Python Library Documentation: module statistics

NAME

statistics - Basic statistics module.

MODULE REFERENCE

https://docs.python.org/3.14/library/statistics.html

The following documentation is automatically generated from the Python source files. It may be incomplete, incorrect or include features that are considered implementation detail and may vary between Python implementations. When in doubt, consult the module reference at the location listed above.

DESCRIPTION

This module provides functions for calculating statistics of data, including averages, variance, and standard deviation.

Calculating averages

Function	Description
mean	Arithmetic mean (average) of data.
fmean	Fast, floating-point arithmetic mean.
geometric_mean	Geometric mean of data.
harmonic_mean	Harmonic mean of data.
median	Median (middle value) of data.
median_low	Low median of data.
median_high	High median of data.
median_grouped	Median, or 50th percentile, of grouped data.
mode	Mode (most common value) of data.
multimode	List of modes (most common values of data).
quantiles	Divide data into intervals with equal probability.

Calculate the arithmetic mean ("the average") of data:

```
>>> mean([-1.0, 2.5, 3.25, 5.75])
2.625
```

Calculate the standard median of discrete data:

```
>>> median([2, 3, 4, 5])
3.5
```

Calculate the median, or 50th percentile, of data grouped into class intervals centred on the data values provided. E.g. if your data points are rounded to the nearest whole number:

```
>>> median_grouped([2
```

, 2, 3, 3, 3, 4]) #doctest: +ELLIPSIS
2.8333333333...
This should be interpreted in this way: you have two data points in the class
interval 1.5-2.5, three data points in the class interval 2.5-3.5, and one in
the class interval 3.5-4.5. The median of these data points is 2.8333...
Calculating variability or spread

Function	Description
pvariance	Population variance of data.
variance	Sample variance of data.
pstdev	Population standard deviation of data.
stdev	Sample standard deviation of data.

Calculate the standard deviation of sample data:
>>> stdev([2.5, 3.25, 5.5, 11.25, 11.75]) #doctest: +ELLIPSIS
4.38961843444...
If you have previously calculated the mean, you can pass it as the optional
second argument to the four "spread" functions to avoid recalculating it:
>>> data = [1, 2, 2, 4, 4, 4, 5, 6]
>>> mu = mean(data)
>>> pvariance(data, mu)
2.5
Statistics for relations between two inputs

Function	Description
covariance	Sample covariance for two variables.
correlation	Pearson’s correlation coefficient for two variables.
linear_regression	Intercept and slope for simple linear regression.

Calculate covariance, Pearson’s correlation, and simple linear regression
for two inputs:
>>> x = [1, 2, 3, 4, 5, 6, 7, 8, 9]
>>> y = [1, 2, 3, 1, 2, 3

, 1, 2, 3]
>>> covariance(x, y)
0.75
>>> correlation(x, y) #doctest: +ELLIPSIS
0.31622776601...
>>> linear_regression(x, y) #doctest:
LinearRegression(slope=0.1, intercept=1.5)
Exceptions

A single exception is defined: StatisticsError is a subclass of ValueError.
CLASSES
builtins.ValueError(builtins.Exception)
StatisticsError
builtins.object
NormalDist
class NormalDist(builtins.object)
| NormalDist(mu=0.0, sigma=1.0)
|
| Normal distribution of a random variable
|
| Methods defined here:
|
| __add__(x1, x2)
| Add a constant or another NormalDist instance.
|
| If *other* is a constant, translate mu by the constant,
| leaving sigma unchanged.
|
| If *other* is a NormalDist, add both the means and the variances.
| Mathematically, this works only if the two distributions are
| independent or if they are jointly normally distributed.
|
| __eq__(x1, x2)
| Two NormalDist objects are equal if their mu and sigma are both equal.
|
| __getstate__(self)
| Helper for pickle.
|
| __hash__(self)
| NormalDist objects hash equal if their mu and sigma are both equal.
|
| __init__(self, mu=0.0, sigma=1.0)
| NormalDist where mu is the mean and sigma is the standard deviation.
|
| __mul__(x1, x2)
| Multiply both mu and sigma by a constant.
|
| Used for rescaling, perhaps to change measurement units.
| Sigma is scaled with the absolute value of the constant.
|
| __neg__(x1)
| Negates mu while keeping sigma the same.
|
| __pos__(x1)